+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Visualizando otras estructuras de datos: Grafos con NetworkX

Introducción

Bienvenidos a esta guía de profundización en la segunda semana, donde exploraremos y visualizaremos datos en forma de grafos y redes utilizando NetworkX, una herramienta esencial para visualizar y analizar estructuras de datos complejas. En esta sesión, nos enfocaremos en los fundamentos de NetworkX, proporcionando una base sólida para aquellos que se inician en la visualización de estructuras de datos complejas. Exploraremos cómo estructurar los datos de manera efectiva y utilizaremos las capacidades de NetworkX para transformar estos datos en visualizaciones claras y significativas. A lo largo de esta guía, aprenderemos a manejar las funcionalidades básicas de NetworkX, desde la creación de grafos simples hasta la implementación de algoritmos básicos para el análisis de redes. Nuestro objetivo es brindarles una comprensión práctica y accesible de cómo NetworkX puede ser una herramienta invaluable en el campo de la visualización de datos, abriendo la puerta a un análisis más profundo y detallado de relaciones y estructuras complejas. ¡Esperamos que esta introducción siga despertando su interés y curiosidad por la visualización de datos y que encuentren en NetworkX un aliado fundamental en su viaje analítico!

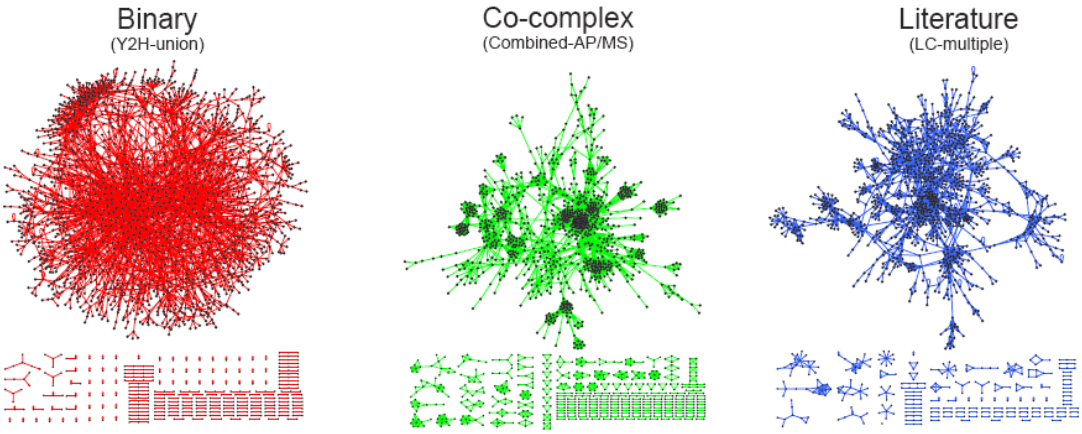
Contenido

Iniciemos nuestra sesión adentrándonos en el concepto de grafos, una estructura fundamental para la visualización de datos. Los grafos son representaciones que nos permiten visualizar las relaciones y conexiones entre distintos elementos, siendo clave en áreas tan diversas como las ciencias sociales, la biología (*ver figura 1*), la informática y más. La importancia de contar con una herramienta de visualización de datos para explorar grafos radica en su capacidad para transformar datos abstractos y complejos en representaciones claras y comprensibles. Con estas herramientas, no solo descubrimos patrones y

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

conexiones ocultas, sino que también facilitamos la interpretación y análisis de las interacciones y estructuras subyacentes. En esta sesión, veremos cómo NetworkX, una biblioteca poderosa y versátil, nos permite desentrañar la complejidad de los grafos y extraer valiosas percepciones de ellos.

Figura 1: High quality binary protein interaction map



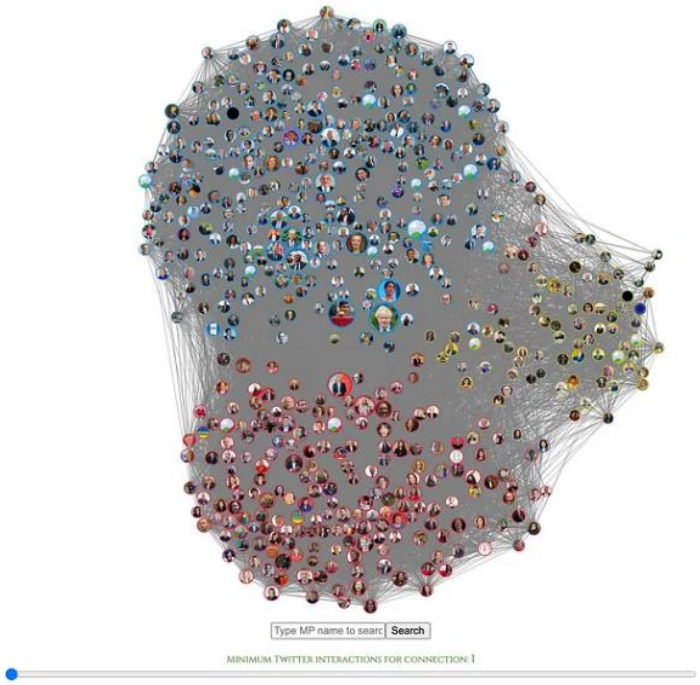
Fuente: Tomado de 'Yeast Interactome Project' por Ye et al, 2008, recuperado de Yeast Interactome Database.

- **NetworkX:**

NetworkX es una biblioteca de Python para analizar, modelar y visualizar redes y grafos complejos. Proporciona herramientas para trabajar con grafos, incluyendo algoritmos para generar grafos aleatorios, estadísticas de grafos, análisis de grafos y visualización de grafos. NetworkX soporta varios tipos de grafos, incluyendo grafos dirigidos y no dirigidos, grafos ponderados y no ponderados, y multigrafos. Es ampliamente utilizado en diversos campos como el análisis de redes sociales, informática, matemáticas y biología. Por ejemplo, la siguiente figura nos muestra una red de relaciones entre los miembros del parlamento británico basado en sus interacciones de Twitter.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 1: High quality binary protein interaction map



Fuente: Tomado de 'Yeast Interactome Project' por Ye et al, 2008, recuperado de Yeast Interactome Database.

Esta gráfica además de dejar en evidencia la importancia de la visualización de grafos está construida con D3.js el framework en el cual se basa NVD3 estudiado en el tema principal de esta semana. Como estos ejemplos mostrados del uso de grafos, hay un sin número de aplicaciones por lo tanto manejar una herramienta como NetworkX es importante para ampliar nuestras habilidades en visualización de datos. Los beneficios que tiene NetworkX son los siguientes:

- **Facilidad de uso:** NetworkX proporciona una interfaz simple e intuitiva para crear y visualizar grafos, haciéndola accesible para usuarios con distintos niveles de experiencia.
- **Personalización:** NetworkX permite la personalización de la apariencia del grafo, incluyendo el tamaño de los nodos, color, etiqueta y estilo de aristas.
- **Escalabilidad:** NetworkX puede manejar grafos grandes con millones de nodos y aristas, lo que lo hace adecuado para el análisis de redes complejas.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Interoperabilidad:** NetworkX puede usarse con otras bibliotecas de Python, como Matplotlib y NumPy, para una visualización y análisis aún más avanzados.
- **Biblioteca de algoritmos:** NetworkX incluye una amplia biblioteca de algoritmos de grafos para tareas como el cálculo del camino más corto, medidas de centralidad y detección de comunidades.
- **Código abierto:** Es un software de código abierto, lo que lo hace disponible gratuitamente para los usuarios y permite el desarrollo y mejora comunitaria.

Guia básica de NetworkX: En esta sesión de lectura vamos a introducir el manejo de NetworkX para la visualización de grafos con un ejemplo práctico. Imagine el siguiente escenario:

- Wayne W. Zachary estudió durante tres años la red social de un club de karate. La red incluía a 34 miembros del club, registrando las interacciones fuera del mismo entre parejas de miembros. Durante el estudio, surgió un conflicto entre el administrador "John A" y el instructor "Mr. Hi", lo que provocó la división del club en dos. La mitad de los miembros creó un nuevo club con Mr. Hi, mientras que los demás encontraron un nuevo instructor o dejaron el karate. Basándose en los datos recopilados, Zachary logró asignar correctamente a casi todos los miembros de los grupos que se unieron tras la división.

Este es un ejemplo muy conocido, nuestro foco principal será visualizar estos datos sin entrar en mucho detalle acerca del objetivo final del trabajo como tal. Así que iniciemos:

1. Cargar datos en NetworkX a partir de un archivo CSV:

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
# Obtener la lista de ejes desde un archivo csv
karate_edgelist = pd.read_csv("https://ona-book.org/data/karate.csv")
# convertir el Dataframe a un grafo de NetworkX
karate = nx.from_pandas_edgelist(karate_edgelist, source = 'from', target = 'to')
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 2: Estructura de los datos de entrada

	from	to
0	Mr Hi	Actor 2
1	Mr Hi	Actor 3
2	Mr Hi	Actor 4
3	Mr Hi	Actor 5
4	Mr Hi	Actor 6
...	...	...
73	Actor 31	Actor 33
74	Actor 31	John A
75	Actor 32	Actor 33
76	Actor 32	John A
77	Actor 33	John A

78 rows × 2 columns

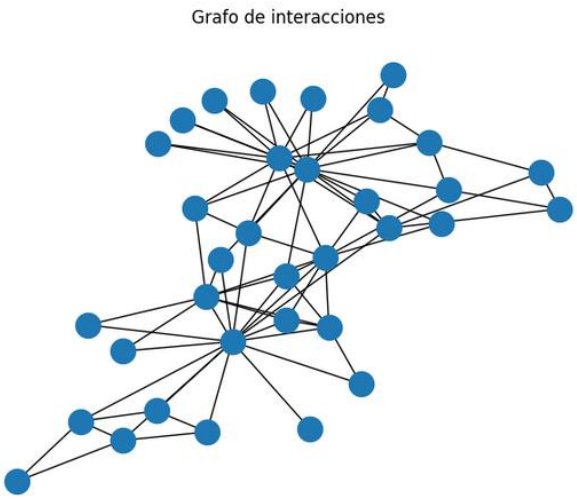
Fuente: Elaboración propia

Como podemos observar la estructura de las interacciones contiene un campo desde donde parte la relación y a que individuo llega, con esto podemos crear el grafo de NetworkX y ver sus propiedades como cuales son los vértices, nodos, si el grafo es dirigido o es un multigrafo, entre otras. Una vez tenemos el objeto grafo, en nuestro caso "*Karate*" basta con llamar a la función *nx.draw* para que nos permita visualizar de manera básica el grafo como sigue usando Matplotlib:

```
# visualizar el grafo usando matplotlib
fig = nx.draw(karate)
# colocar titulo a la figura
plt.title('Grafo de interacciones')
# mostrar el grafo
plt.show()
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 3: Visualización del grafo Karate



Fuente: Elaboración propia

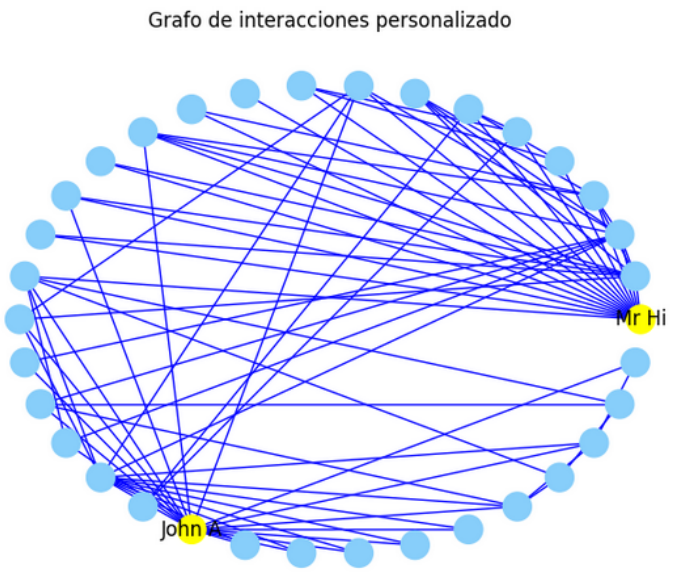
De manera directa tenemos visualizado nuestro grafo de interacciones. Sin embargo, no tiene muchas opciones de personalización, a menos que trabajemos directamente con Matplotlib. Afortunadamente, para efectos prácticos, NetworkX tiene unas funciones que nos permiten agregar personalizaciones (*nx.draw_networkx* y *nx.draw_circular*). Por ejemplo, vamos a agregar color a los nodos y organizar nuestro grafo de manera circular.

```
# recorrer los nodos y solo seleccionar los nombres
# de los nodos que queremos cambiar de color
node = list(karate.nodes)
labels = [i if i == "Mr Hi" or i == "John A" else ""
                for i in karate.nodes]
# construir un nuevo diccionario de nodos pero esta vez
# con los nombres de sólo los nodos objetivo
nodelabels = dict(zip(node, labels))
# con base en esto creamos una nueva lista de colores
colors = ["yellow" if i == "Mr Hi" or i == "John A" else
"lightskyblue" for i in karate.nodes]
# visualizamos nuestro grafo en forma circular con los nuevos colores
# y el color eje lo podemos cambiar también
nx.draw_circular(karate, labels = nodelabels, node_color = colors,
edge_color = "blue")
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
plt.title('Grafo de interacciones personalizado')
plt.show()
```

Figura 4: Grafo de interacciones con opciones de personalización



Fuente: Elaboración propia

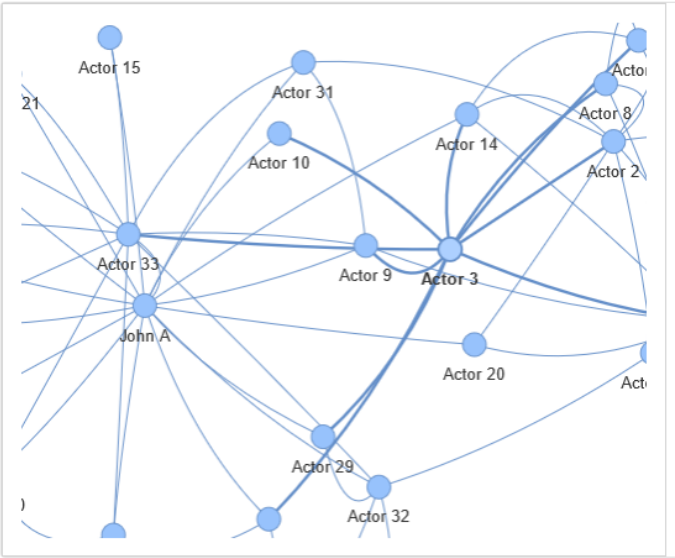
Con NetworkX tenemos más opciones de personalización que dejamos a voluntad del estudiante. Tales opciones incluyen la posibilidad de agregar etiquetas a los ejes y vértices, cambiar tamaños, estilos de aristas, entre otras. Por otro lado, como ya se ha visto las ventajas y las desventajas que hay entre las visualizaciones estáticas y las interactivas. NetworkX a pesar de ser una librería de visualizaciones estáticas permite la integración con librerías de carácter interactivo para grafos exportando la información (GraphViz, D3.js, Bokeh y plotly) o de integración directa como nxviz o Pyviz. Por tal motivo, agregamos un ejemplo usando Pyviz:

```
from pyvis.network import Network
from IPython.display import HTML
# create pyvis Network object
net = Network(height = "500px", width = "600px", notebook = True,
              cdn_resources='remote')  #  remote  si  esta  en  google
colab
net.from_nx(karate) # importar el grafo karate
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
# alternativa directa si estamos trabajando en local
#net.show('out1.html', local='in_line', notebook=True)
# para visualizar en colab, guardar el html correctamente
net.write_html(name='out1.html',      local=True,      notebook=True,
open_browser=True)
# y abrir con IPython
with open('out1.html', 'r') as file:
    html_content = file.read()
# visualizar el contenido HTML en google colab
HTML(html_content)
```

Figura 5: Grafo de interacciones interactivo con Pyvis



Fuente: Elaboración propia

Con este ejemplo finalizamos nuestra lectura de profundización, animamos al estudiante a seguir explorando estas dos nuevas herramientas de visualización de grafos y así obtener todo el conocimiento en este tópico especifico de visualización.

Conclusiones

En esta guía de profundización se recalcó la importancia de manejar herramientas adicionales para manejar diferentes tipos de datos, en este caso los

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

grafos, importantes en el análisis y visualización de información. Específicamente, se introdujo a NetworkX como herramienta de visualización estática de grafos pero también se usó Pyvis la cual está integrada directamente con NetworkX y permite una visualización de grafos interactiva. Esperamos que esta lectura haya sido de mucho interés y que haya alcanzado el objetivo final que es proporcionar al estudiante más herramientas en la visualización de datos.

Referencias

Keith McNulty, 2022, Handbook of Graphs and Networks in People Analytics, recuperado el 20 de enero de 2022 de https://ona-book.org/viz-graphs.html#visualizing-graphs-in-python

NetworkX Developers. (n.d.), Tutorial, Recuperado el 20 de enero de 2024 de https://networkx.org/documentation/stable/tutorial.html

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Plotly, Bokeh y NVD3.js: Guía básica

Introducción

Bienvenidos a esta primera guía de profundización de la semana 2, aquí introduciremos la creación de gráficos 3D e interactivos con Plotly, Bokeh y NVD3. A su vez, iremos abordando cómo definir la estructura de datos necesaria para que cada librería convierta datos en visualizaciones interactivas amigables a la vista. Dejaremos abierta la posibilidad para que experimenten cual librería se adapta más a su estilo de programación y esperamos despertar un poco más el interés en el amplio mundo de la visualización de datos.

Contenido

1. Visualización de datos en 3D:

Si bien existen una gran variedad de librerías para la visualización de datos, algunas de ellas no cuentan con visualización en 3D. En el temario de esta semana, para la creación de gráficos 3D, hicimos uso de una librería abordada en la primera semana (Matplotlib). Ya que está nos permite construir gráficos en 3D de una manera bastante amigable y sencilla. Por lo tanto, retomando el ejemplo de la agencia inmobiliaria, podemos crear el gráfico 3D en un par de líneas de código Python:

```
import numpy as np
import matplotlib.pyplot as plt
# Comando mágico en Jupyter notebook para pintar el gráfico en
# la misma celda
%matplotlib inline
# Se define en conjunto de datos a visualizar
house_dataset = np.array([[1, 60, 1000],[1, 63, 1050],
                          [1, 66, 1300],[1, 70, 1400],
                          [1, 65, 1100],[1, 61, 1050],
```

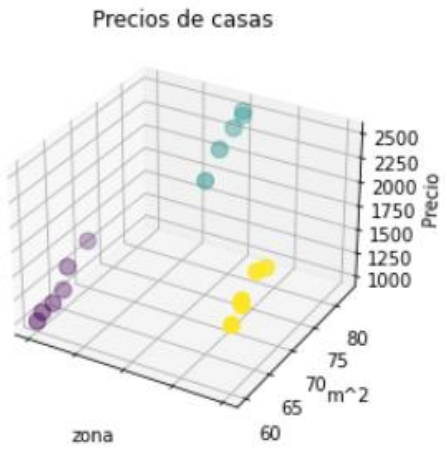
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
                          [2, 75, 2100],[2, 78, 2300],
                          [2, 83, 2500],[2, 83, 2450],
                          [2, 81, 2400],[3, 60, 1600],
                          [3, 62, 1700],[3, 65, 1900],
                          [3, 62, 1750],[3, 67, 1850]])
# Se crea una figura con un tamaño especificado en el figsize
fig = plt.figure(1, figsize=(4, 4))
# se crea el bosquejo del gráfico, especificando que tiene una
proyección en
# 3 dimensiones
ax = fig.add_subplot(111, projection="3d")
# Se utiliza el gráfico de dispersión, especificando las 3
dimensiones y se especifica que el primer valor del conjunto de datos
los clasifica (zona)
ax.scatter(house_dataset[:, 0],
    house_dataset[:, 1],
    house_dataset[:, 2],
    c=house_dataset[:, 0], s=80,)
# Se agrega fondo blanco al gráfico y etiquetado de ejes
ax.axes.set_facecolor(color="White")
ax.set_title("Precios de casas")
ax.set_xlabel("zona")
ax.xaxis.set_ticklabels([])
ax.set_ylabel("m^2")
ax.set_zlabel("Precio");
```

Es de resaltar que se importa el módulo numpy de computación científica, el cual permite crear la matriz con los datos de prueba, y el módulo de Matplotlib, que nos permite crear el gráfico 3D. Note que como en la semana anterior, se usaron marcadores y colores para obtener un mejor gráfico. Además, se usó un comando mágico para que el graficó generado por Matplotlib se viera dentro del Jupyter Notebook. Cómo resultado de la ejecución tendremos la siguiente gráfica:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 1: Gráfico 3D en Matplotlib



Fuente: Elaboración propia.

2. Gráficos interactivos con Plotly:

Una de las virtudes principales de Plotly, es que nos permite crear un gráfico interactivo con muy pocas líneas de código, aunque esto nos puede jugar en contra debido a que a veces no nos deja personalizar del todo ciertas características de los gráficos. Por ejemplo, bastan solo 4 líneas de código para graficar los datos referentes al precio del cierre de 6 diferentes acciones:

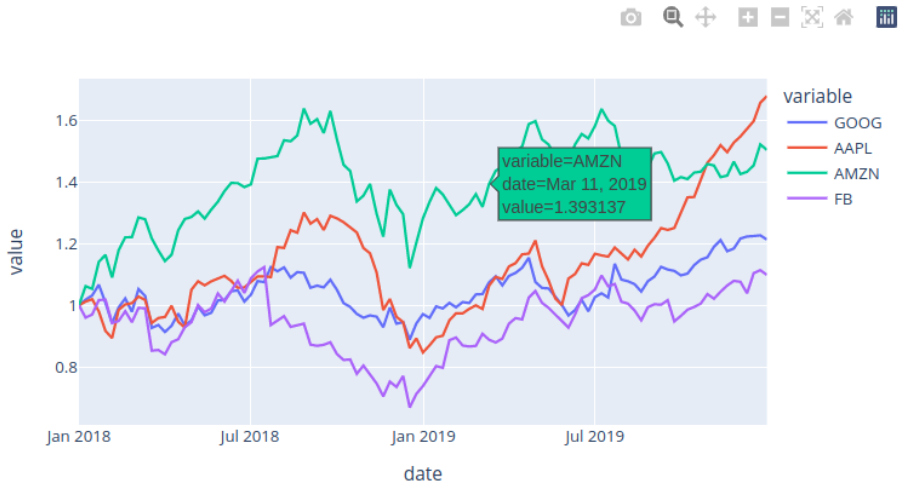
```
import plotly.express as px
# Cargamos el dataframe de la misma librería de plotly
df = px.data.stocks()
# Especificamos un gráfico de líneas conde el eje "x" tendrá las
fechas y el eje "y" los diferentes valores de las acciones.
fig = px.line(df, x='date', y=df.columns[1:5],width=700,
height=400)
# Se pinta la gráfica
fig.show()
```

Note que importamos la librería Plotly, de la cual hacemos uso para importar el conjunto de datos a graficar. A su vez, se puede evidenciar que Plotly no necesita que se le especifique directamente los valores a graficar en las variables *"x"* e *"y"*. Simplemente le pasamos como entrada el *DataFrame* llamado *"df"*, y especificamos que la columna *"date"* del *DataFrame* será el eje *"x"*, y la columna

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

"*y*" tomará los valores de las diferentes columnas. Como resultado tendremos el siguiente gráfico interactivo:

Figura 2: Gráfico Interactivo en Plotly.



Fuente: Elaboración propia.

Otra de las principales ventajas que tiene Plotly es que podemos crear un pequeño *dashboard* con pocas líneas de código, por ejemplo, si queremos correlacionar los precios de las acciones, podemos realizarlo mediante el siguiente código en Python:

```
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
app = Dash(__name__)
# Se agrega una capa la cual contiene la lista desplegable
# Se definen las opciones disponibles a seleccionar
app.layout = html.Div([
    html.H4('Análisis de matriz de dispersión'),
    dcc.Dropdown(
        id="dropdown",
        options=['GOOG', 'AAPL', 'AMZN', 'FB'],
        value=['GOOG', 'AMZN'],
        multi=True
    ),
    dcc.Graph(id="graph"),
])
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
@app.callback(
    Output("graph", "figure"),
    Input("dropdown", "value"))
def update_bar_chart(dims):
    # Cargamos el dataframe de la misma librería de plotly
    df = px.data.stocks()
    fig = px.scatter_matrix(
        df, dimensions=dims)
    return fig
app.run_server(debug=True)
```

En este ejemplo introducimos la librería Dash, la cual está desarrollada para conectarse con Plotly, y permitir agregar a nuestros gráficos más componentes interactivos como lo son en este caso las listas desplegables (*Fig. 2*). Mediante Dash creamos una aplicación, la cual contiene un título, una lista desplegable que permite seleccionar 4 acciones diferentes. Luego añadimos un *Graph*, que nos permitirá crear un espacio de gráfico interactivo.

Figura 3: Dashboard de correlación en Plotly

Fuente: Elaboración propia.

Note que dentro de nuestro código definimos una función *callback*, estás funciones son esenciales al momento de crear un gráfico interactivo. Lo que

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

hacen es dada una entrada de datos, retornar una nueva gráfica con el ajuste a los datos. Esta es una de las ventajas de usar conjuntos de datos como los *DataFrame*, ya que cuentan con información extra como nombre de columnas e índices, lo cual permite fácilmente decidir que se quiere graficar. Por ejemplo, si el usuario objetivo quiere ver la relación de las acciones de GOOG y FB, basta con seleccionar esas dos acciones de la lista desplegable, que la función callback se encargará de actualizar la gráfica.

3. Gráficos interactivos con Bokeh:

Bokeh es una de las librerías más utilizadas al momento de generar gráficos interactivos en Python, su mayor flexibilidad en la creación de gráficos es pieza clave, y a la vez es uno de sus retos, ya que a diferencia de Plotly, se necesita configurar un par de cosas extra para tener un gráfico interactivo básico. Por ejemplo, a diferencia de Plotly, si queremos visualizar los valores exactos de las acciones en el tiempo al pasar el cursor sobre ellas, debemos especificar a nivel del código dicha funcionalidad. Por ejemplo, en el siguiente bloque de código creamos un gráfico con dos precios de acciones:

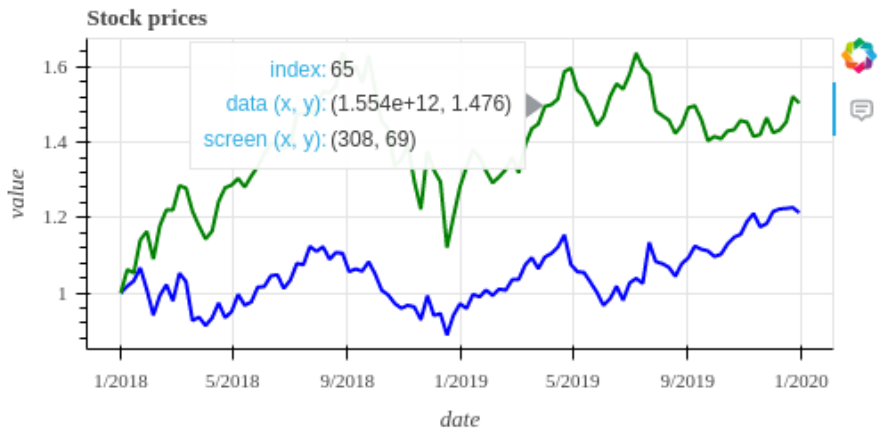
```
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.models import ColumnDataSource
from bokeh.models import HoverTool
from bokeh.models import DatetimeTickFormatter
# Cargamos el conjunto de datos desde plotly
df = px.data.stocks()
# Es necesario formatear la fechas para que bokeh las interprete
como fechas
df["date"] = pd.to_datetime(df["date"])
# Se transforma el DataFrame a ColumnDataSource para aumentar la
eficiencia de renderización
source = ColumnDataSource(df)
# Se crea el gráfico y se configura un título y se etiquetan los
ejes.
p = figure(title="Stock prices", x_axis_type="datetime",
x_axis_label="date", y_axis_label="value",
          tools=[HoverTool()], max_width=500, height=250,)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
# Se añaden los gráficos de líneas
p.line(x="date", y="GOOG", color="blue", line_width=2,
source=source)
p.line(x="date", y="AMZN", color="green", line_width=2,
source=source)
show(p)# Se muestra el gráfico
```

Note que a los gráficos no se les envía directamente los valores para el par de ejes *x* e *y*, en vez de eso, se adjunta un *source* el cual es el conjunto de datos a graficar (*ColumnDataSource)*, y haciendo uso del nombre de las columnas, se especifica qué columna se tome para ser representada por el eje *x* y que otra columna sea representada por el eje *y*. Una de las ventajas al usar Bokeh, es que a un mismo gráfico (llamado figura en Bokeh), le podemos añadir diferentes tipos de representaciones gráficas. En la Figura 4 podemos ver el gráfico generado por el bloque de código anteriormente mencionado. Note que la etiqueta que sale al pasar el cursor sobre la gráfica no es muy entendible a primera vista, en comparación a la etiqueta que genera por defecto Plotly.

Figura 4: visualización básica con Bokeh



Fuente: Elaboración propia.

Se puede observar que por defecto, el gráfico no permite identificar a qué acción pertenece cada gráfico de línea. Por consiguiente, vamos a explorar un poco más las opciones de configuración que nos brinda Bokeh. Para ello, definiremos una etiqueta para poder identificar a qué acción pertenece cada gráfico. A su vez,

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

modificaremos la etiqueta que pinta Bokeh por defecto para que nos resulte más amigable a la vista.

```
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.models import ColumnDataSource
from bokeh.models import HoverTool
from bokeh.models import DatetimeTickFormatter

# Cargamos el conjunto de datos desde plotly
df = px.data.stocks()
# Es necesario formatear la fechas
df["date"] = pd.to_datetime(df["date"])
# Se transforma el DataFrame a ColumnDataSource para aumentar la
eficiencia de renderización
source = ColumnDataSource(df)
# Definimos la apariencia de la leyenda interactiva a mostrar y
definimos la columna date
# como formato de fecha para que pueda ser interpretado
correctamente por bokeh
ht = HoverTool(
    tooltips=[
        ('date','@date{%F}'),
        ('GOOG','@GOOG'),
        ('AMZN','@AMZN'),
    ],
    formatters={
        '@date':'datetime',
    })
# Se crea el gráfico y se configura un título y se etiquetan los
ejes.
p = figure(title="Stock prices", x_axis_type="datetime",
x_axis_label="date", y_axis_label="value",
          tools=[ht, 'wheel_zoom, zoom_in, save, reset, pan'],
max_width=500, height=250)
# Se añaden los gráficos de líneas
p.line(x="date", y="GOOG", color="blue", line_width=2,
source=source, legend_label="GOOG")
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
p.line(x="date", y="AMZN", color="green", line_width=2,
source=source, legend_label="AMZN")
show(p) # Se muestra el gráfico
```

Lo que hicimos fue interactuar con el panel de *tools* de Bokeh, el cual nos permite especificar los datos a mostrar en el gráfico interactivo. Para este caso especificamos 3 datos importantes: La fecha, la cual tenemos que especificar mediante el *formatters* que es de tipo *datetime,* para que Bokeh la pueda interpretar de forma correcta y no coloque un número (*Fig. 4*). A su vez, especificamos los valores del precio de cierre de las dos acciones (GOOG y AMZN). Note que para identificar cada gráfico lineal, lo que hicimos fue agregar a cada uno la propiedad *legend_label*. Obteniendo el siguiente gráfico:

Figura 5: Interacción con las herramientas de visualización.

Fuente: Elaboración propia.

4. Gráficos interactivos con NVD3.js:

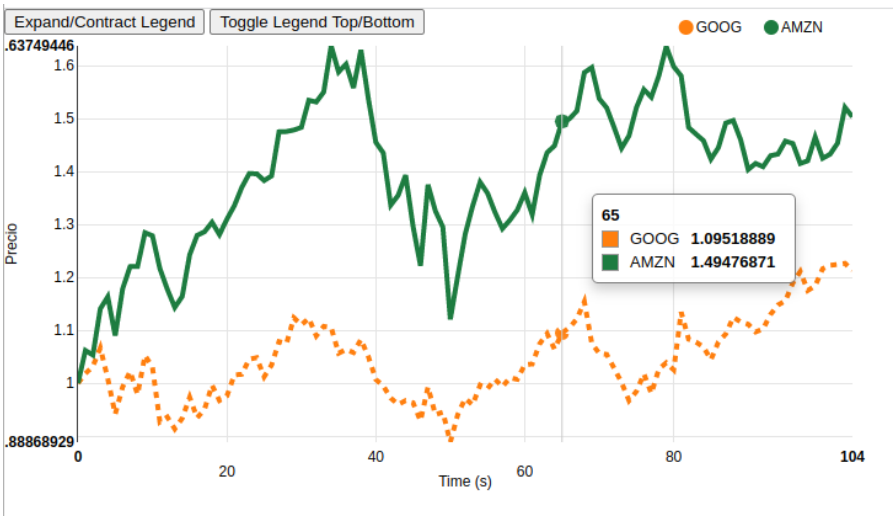
Como abordamos en el temario de la semana 2, NVD3 es una librería desarrollada en JavaScript la cual agrupa diferentes tipos de gráficos interactivos desarrollados en D3.js (tal vez la herramienta más poderosa para el diseño de gráficos interactivos). NVD3 nos brinda una abstracción, una capa más alta de programación de gráficos en JavaScript, para no tener que incurrir en el bajo nivel y la gran dificultad que puede implicar el crear un gráfico en D3.js. Para poder iniciar a graficar con NVD3.js es necesario contar con las dependencias descritas en la librería. Además de tener conocimientos básicos en HTML y CSS, para

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

poder visualizar en una página web los gráficos generados. Cabe resaltar que se tienen que tener conocimientos básicos de D3.js para poder generar ciertas interacciones. Pero como tal, NVD3.js nos brinda ya plantillas con diferentes tipos de gráficos interactivos, una gran ventaja en comparación de D3.js, pues en este último es necesario programar absolutamente todo.

Debido a la complejidad en la programación del código para generar un gráfico interactivo (*Fig. 6*), el código empleado será anexado al documento y nos enfocaremos en los detalles relevantes del código.

Figura 6: Visualización interactiva mediante NVD3.js.



Fuente: Elaboración propia.

Para el ejemplo en concreto, colocamos dentro del *script* HTML los datos a graficar. En un ambiente productivo, estos podrían ser retornados mediante un consumo a un API. Al fijarnos en el código veremos que el gráfico se añade a un *div* de HTML mediante la librería D3.js, a este se le pasan los datos en un formato de lista, donde cada posición representa un archivo JSON el cual contiene pares clave, valor que identifican la configuración de cada gráfico lineal a ser representado. Hasta ahora, vemos que utilizamos la librería D3.js para insertar el gráfico en la web. Entonces: ¿Cuál es el beneficio de NVD3.js? Este se puede identificar al momento de crear el gráfico a representar. NVD3.js cuenta con una lista de modelos ya programados en JavaScript en los cuales solo necesitamos configurar ciertos parámetros de entrada.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Note que para este ejemplo en concreto, seleccionamos de los modelos de NVD3.js un gráfico *lineChart*. Una vez definido el tipo de gráfica, lo visualizamos mediante D3.js, le pasamos por medio del método *datum* los datos a graficar, y mediante el método *call*, pasamos el gráfico *lineChart* definido anteriormente.

Conclusiones

Comprender los conceptos básicos para la visualización de gráficos interactivos en Plotly, Bokeh y NVD3.js: Tenga presente que tipo de gráfico quiere representar y cuál de las librerías abordadas le permitirá hacerlo en el menor tiempo posible. Se recomienda generar un gráfico base con Plotly, debido a que brinda la forma más sencilla de crear una visualización interactiva. Si quiere tener un poco más de libertad al diseñar, puede usar Bokeh. Y si quieres volverte un experto en visualización, ¡navega en el mundo de NVD3.js o D3.js!

Referencias

Novous Partners, (n.d.), NVD3 Re-usable charts for d3.js, recuperado el 14 de enero de 2024 de https://nvd3-community.github.io/nvd3/examples/documentation.html

Plotly, Inc. (n.d.), Dash Core components, Recuperado el 14 de enero de 2024 de https://dash.plotly.com/dash-core-components

Anaconda, Inc., and Bokeh Contributors, Bokeh First steps, (n.d.), Recuperado el 15 de enero de 2024 de https://docs.bokeh.org/en/latest/docs/first_steps.html#first-steps

Anexos

Anexo a este material se encuentra un archivo llamado d3_ stock.zip. Dentro de este encontrará una carpeta *build* la cual contiene los archivos necesarios para hacer uso de la librería NVD3.js. Para poder visualizar el gráfico interactivo, basta con abrir en el navegador de su preferencia el archivo *stock.html*. Para explorar el código, es necesario abrir el archivo en un editor de texto.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Introducción a la visualización de datos interactiva

Introducción

Introducción a gráficos 3D e interactivos: Plotly, Bokeh y NVD3

¡Bienvenido a la segunda semana de fundamentos de la visualización de datos! En esta semana, nos adentraremos en el mundo de la visualización interactiva y destacaremos la relevancia de poder interactuar con datos a partir de gráficas, lo cual nos ayudará a no solo tener un visual general de los datos (visualizaciones estáticas), sino que también nos ayudará a profundizar en los datos y poder agregar más información en una misma visualización.

Palabras clave

Visualización de datos, Visualización interactiva, Gráficas interactivas, Comunicación visual, D3.js.

Reto

Cuando nos encontramos con grandes volúmenes de datos, los cuales contienen varías características de importancia. Nos encontramos con el reto de ¿Cómo reflejar las características más importantes en un gráfico sin que tenga una alta complejidad visual? Descubramos el poder de las visualizaciones interactivas.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Desarrollo

La importancia de entender cómo visualizar datos a gran escala

En la actualidad, difícilmente nos encontramos con una compañía que no explote la visualización de datos para la toma de decisiones favorables al negocio. Por lo cual surge la necesidad de entender cómo visualizar datos de forma correcta, ya que muchas veces se puede incurrir en el error de generar visualizaciones que son entendibles a simple vista, pero que no generan valor monetario al negocio.

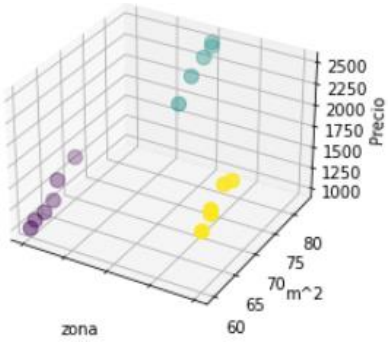
Cuando hablamos de datos recolectados por compañías en diferentes fases de sus procesos, solemos hablar de grandes volúmenes de datos, lo cual suele hacer más difícil la tarea de generar valor al visualizarlos. Es por esto que, además de las gráficas estáticas en 2D, surgen otros tipos de visualizaciones que nos permiten resaltar de una mejor forma las características de los datos.

1. Visualizaciones en 3D:

Las visualizaciones en tres dimensiones normalmente nos ayudan a ver ciertas correlaciones en los datos que no logramos apreciar en visualizaciones 2D. A su vez, nos permiten separar/etiquetar tipos de datos que cumplen con ciertas características. De la misma forma que se correlacionan dos características en visualizaciones 2D, las visualizaciones 3D nos permiten agregar una característica más a correlacionar, lo que nos permite tener un panorama más amplio visualmente. Por ejemplo, si una agencia inmobiliaria quiere agregar valor a los reportes generados a potenciales compradores. Mediante un gráfico 3D (*Fig. 1*), puede correlacionar ciertos datos de interés del cliente, como lo son los metros cuadrados de la propiedad, la zona donde se encuentra y el valor del inmueble. Así, el posible comprador puede discernir que ciertas zonas pueden tener un valor de metro cuadrado más alto, lo que le puede ayudar a tomar una decisión.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 1: Precios de viviendas

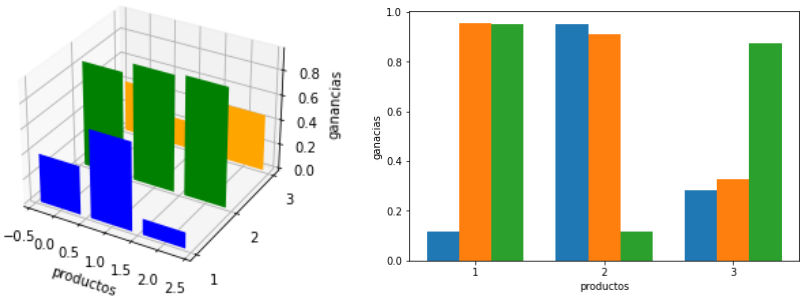


Fuente: Elaboración propia.

Las visualizaciones en 3 dimensiones nos acompañan a diario. Nuestra misma visión ocular nos permite distinguir de los objetos su altura, anchura y profundidad. Esto nos permite tener una vista más amplia al poder ver todos los ángulos de un objeto. Así mismo, la representación de datos en 3D nos brinda ventajas que no se podrían obtener con una visualización 2D, como lo es el ampliar el espacio de entendimiento de los datos y poder generar gráficos interactivos que permitan al usuario observar relaciones en los datos desde distintos ángulos.

No obstante, las visualizaciones en más de dos dimensiones, suelen tener una complejidad más alta para su comprensión, por lo cual es importante determinar si en realidad aportaría o no valor la visualización 3D con base al grupo objetivo al cual se quiere encaminar. En algunos casos, es mejor optar por una visualización 2D (*Figura 2*).

Figura 2: Representación de datos en diferentes dimensiones.



Fuente: Elaboración propia.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Note que en la visualización 3D (*Figura 2*), no se puede apreciar visualmente el valor de algunos productos debido a que estos quedan ofuscados por los demás gráficos de barras. Esto se podría solucionar mediante un gráfico de barras agrupado en una visualización 2D, haciendo el gráfico más entendible para el usuario objetivo.

2. Gráficos interactivos:

Los gráficos interactivos suelen aportar mucha más información que un gráfico estático en 2D o 3D, ya que este le permite al usuario objetivo interactuar con la visualización y encontrar detalles relevantes que a simple vista en un gráfico estático no se podrían ver, o que lo harían muy denso de entender al contener mucha información. Exploremos las principales características que contiene el panel de configuración de un gráfico interactivo y las cuales son de vital utilidad para que el usuario objetivo pueda interactuar de forma amigable.

- **Ampliar la gráfica:**
  Permite que el usuario pueda seleccionar una parte de la gráfica por la cual tenga mayor interés y se realice un *zoom,* lo que puede ser útil para mostrar mejor la variación de valores cercanos. O si estos están etiquetados por colores, permite aclarar a qué clase pertenecen.

- **Enfocar datos:**
  Se permite hacer un enfoque en los valores a destacar en la gráfica, lo que realza los valores encerrados dentro de un cuadro o un trazo libre realizado por el usuario objetivo. Todos los valores fuera de dicho trazo, son visualizados de forma clara, para alejarlos de la vista del usuario.

- **Desplazarse sobre los ejes:**
  Esta funcionalidad es de bastante utilidad cuando se hace *zoom* sobre el gráfico y se desea desplazar por la línea de tiempo de los ejes del gráfico.

- **Descargar captura:**
  Permite que el usuario descargue una imagen del estado actual del gráfico. Lo que implica que el usuario pueda aplicar los filtros anteriormente

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

mencionados y luego descargar esa gráfica mostrando únicamente los datos que se consideren de interés.

- **Reiniciar los ejes:**

Permite volver al gráfico inicial. Esto es de gran ayuda por si se aplican filtros errados al gráfico, o si queremos ver nuevamente la totalidad de los datos. Además de las características resaltadas anteriormente. Existen otras visualizaciones que sirven como panel para filtrar los datos de las gráficas. Estas serán explicadas en detalle en la lectura de profundización.

3. Herramientas para la graficación interactiva

- **Gráficos interactivos con Plotly:**

Plotly es una librería que nos permite hacer gráficos interactivos a partir de código en Python. Esta librería funciona mediante objetos de Python los cuales son serializados en archivos JSON para ser entendidos posteriormente por la librería de JavaScript de Plotly. Antes de adentrarnos en los gráficos interactivos que podemos realizar en Plotly. Es importante definir que la estructura de datos más utilizada para crear gráficos interactivos en Plotly es el *DataFrame* (*Tabla 1*), el cual se podría asemejar a una tabla de datos, la cual contiene *m* filas (las cuales tienen un índice que las identifica) y *n* columnas con base a la cantidad de características que tenga cada fila.

Tabla 1 : Representación de un DataFrame

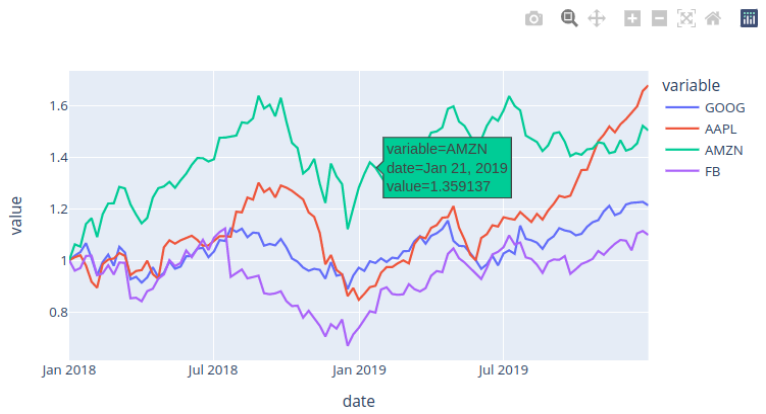
	date	GOOG	AAPL	AMZN	FB	NFLX	MSFT
0	2018-01-01	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
1	2018-01-08	1.018172	1.011943	1.061881	0.959968	1.053526	1.015988
2	2018-01-15	1.032008	1.019771	1.053240	0.970243	1.049860	1.020524
3	2018-01-22	1.066783	0.980057	1.140676	1.016858	1.307681	1.066561
4	2018-01-29	1.008773	0.917143	1.163374	1.018357	1.273537	1.040708

Fuente: Elaboración propia.

Para este ejemplo en particular utilizaremos un conjunto de datos el cual nos brinda la misma librería y que se refiere al precio de cierre de 6 diferentes acciones. Note que un primer gráfico que podemos realizar es el de ver cómo fluctúa el precio de cada acción en el transcurso del tiempo (*Fig. 3*)

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 3: Precios de cierre de diferentes acciones año 2018/2019.



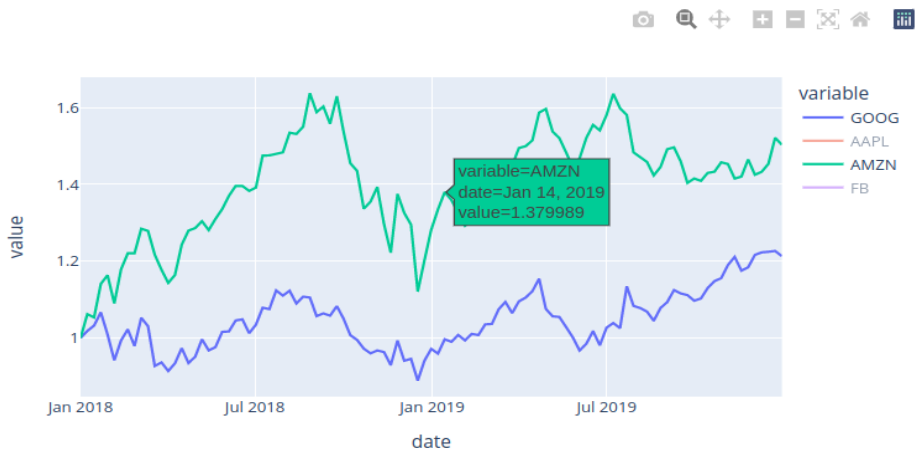
Fuente: Elaboración propia.

Podemos observar que en la margen superior derecha contamos con el panel de configuración del gráfico, el cual nos ayuda a interactuar con los datos. A su vez, una característica que traen los gráficos generados por medio de un *DataFrame*, es que al desplazar el cursor por la gráfica, este nos muestra el valor exacto en ese punto de tiempo, algo que a simple vista no se podría determinar, y una gran ventaja para con las visualizaciones estáticas (no interactivas).

Otra característica con la que cuentan los gráficos por defecto en Plotly, es que si los datos graficados cuentan con una etiqueta que los diferencie, existe la opción de ocultar ciertos valores para enfocarse solo en los de interés. Por ejemplo, si queremos ver sólo la variación del precio de las acciones de GOOG y AMZN, basta con dar *click* en la etiqueta de las acciones a ocultar en la parte derecha del gráfico interactivo (*Fig. 3*), y obtendremos un nuevo gráfico (*Fig. 4*) el cual solo contará con los datos en los que nos queremos enfocar.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 4: Precios de cierre de GOOG y AMZN año 2018/2019, ilustrado con Plotly.



Fuente: Elaboración propia.

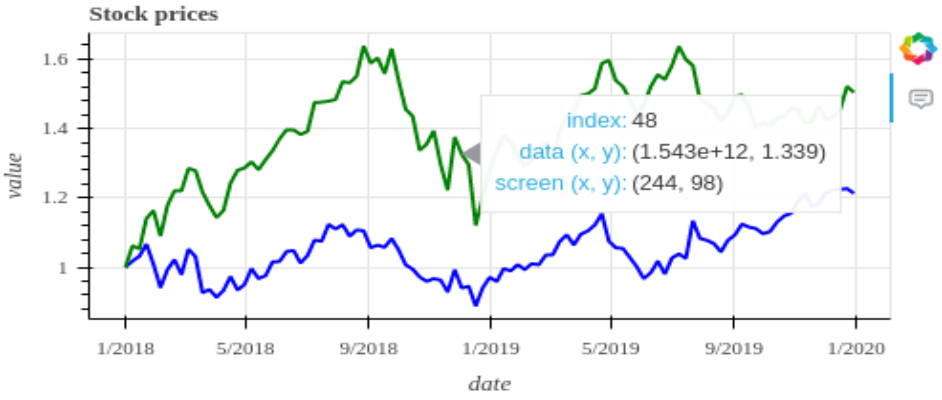
- **Gráficos interactivos con Bokeh:**

Bokeh es la librería preferida al momento de realizar *dashboards* y gráficos interactivos. Esta librería está hecha en Python, y al igual que Plotly, nos permite realizar gráficos interactivos compatibles con la web sin escribir líneas de código en JavaScript, ya que nos permite exportar las gráficas que creamos en dicho lenguaje de programación. Bokeh combina dos elementos principales: La librería de Python la cual estaremos utilizando en la guía de profundización, y una librería de JavaScript llamada BokehJS, la cual trabaja por debajo de nuestro código para poder visualizar los gráficos interactivos.

A diferencia de Plotly, la estructura de datos principal de Bokeh es el *ColumnDataSource*, el cual es una abstracción propia de Bokeh, la cual al ser usada, nos da una mejor eficiencia en la visualización/renderización de las gráficas construidas. Si bien Bokeh permite como entrada listas, arrays, e inclusive *DataFrames,* se recomienda que estos tipos de datos sean convertidos a *ColumnDataSource* para aumentar el rendimiento de la librería. Basándonos en el conjunto de datos de la Tabla 1, podemos construir con Bokeh (*Fig. 5*) la misma representación gráfica de la Figura 4. Solo que como veremos más adelante, la complejidad con la que se llega al mismo resultado, es más alta al utilizar Bokeh.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 5: Precios de cierre de GOOG y AMZN año 2018/2019, ilustrado con Bokeh.



Fuente: Elaboración propia.

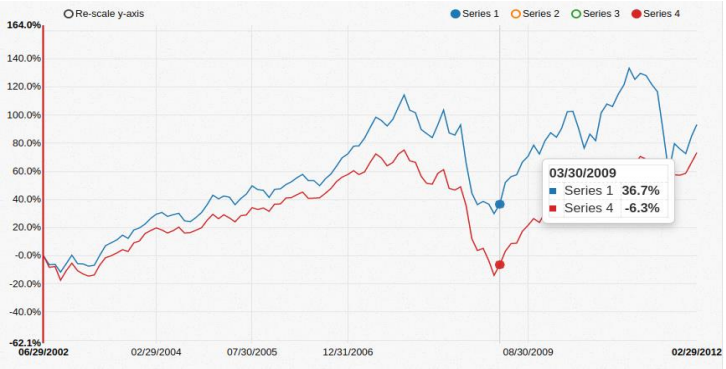
Al igual que Plotly, en Bokeh se cuenta con un panel de configuración, el cual permite al usuario interactuar con la gráfica. Cabe resaltar que al generar un gráfico interactivo en Bokeh, este se abre en el navegador predeterminado. Para poder ver el gráfico por ejemplo dentro de Jupyter notebook, hace falta una configuración extra.

- **Gráficos interactivos con NVD3:**

No podemos hablar de NVD3.js sin relacionarlo con D3.js, la más potente librería en JavaScript para la creación de gráficos interactivos. El proyecto NVD3.js surgió como un intento de reutilizar gráficos de D3.js para mitigar la complejidad de crearlos desde cero. Y es que por ser D3.js una librería poderosa para la creación de gráficos interactivos, su curva de aprendizaje es compleja, se necesita mucha experiencia para poder crear un gráfico que en Plotly o Bokeh se haría en un par de líneas de código. Una de las grandes virtudes de utilizar NVD3.js para la creación de gráficos interactivos, es que por estar escrito en JavaScript, se cuenta con una libertad de desarrollo más amplia. Lo que nos puede llevar a crear visualizaciones más detalladas (*Fig. 6*). La experiencia del usuario objetivo al desplazarse sobre la gráfica es atrapante, debido a los efectos visuales que ofrece JavaScript.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 6: Visualización de serie de tiempo en NVD3.js



Fuente: Elaboración propia.

Cabe resaltar que la estructura de datos de la que se alimenta NVD3.js son los archivos JSON (Java Script Object Notation), los cuales contienen una colección de pares de clave-valor y listas de datos ordenadas, por lo cual son legibles a simple vista desde un editor de texto. Una de las preguntas que surgen al finalizar el temario, es: ¿Cuál librería debería usar para cada caso de visualización interactiva? Si bien, para la mayoría de los casos se pueden utilizar las tres librerías. Existen limitantes en algunas. Por ejemplo, en Bokeh no se pueden realizar gráficas en 3D. O si hablamos de grandes cantidades de datos a visualizar, es mejor utilizar Bokeh o NVD3.js, ya que Plotly al emplear *DataFrames* como fuente de origen de los datos, suele ser más lenta al renderizar los gráficos. Y si lo que se desea es aprender a construir gráficos en el menor tiempo posible, NVD3.js sería la opción a no elegir, por su compleja curva de aprendizaje.

Conclusiones

Resumen de la introducción a la visualización de datos interactiva.

Al concluir esta exploración de visualizaciones en 3D e interactivas, es necesario resaltar que se adquirieron las siguientes bases:

- **Comprender la importancia de la visualización 3D**: Reconocer que la visualización de datos en 3 dimensiones puede ser una forma óptima de darle valor a los datos. No obstante, se debe tener claro que no se puede

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

abusar de dicha visualización, ya que puede que no aporte valor, sino que confunda al visualizador.

- **Reconocer la importancia de las visualizaciones interactivas en grandes volúmenes de datos**: Reconozca que no siempre las visualizaciones 2D o 3D van a poder darle toda la información que necesita graficar. Por esto, es necesario hacer uso de las gráficas interactivas, con las cuales podremos dar a detalle valores relevantes, sin dejar al ojo humano la aproximación del valor. Y permitimos que el usuario, al interactuar con el gráfico, pueda encontrar otras relaciones que a simple vista no se logran ver.

- **Familiarizarse con las principales librerías de visualización de datos interactivos (Plotly, Bokeh y NVD3.js)**: Recuerde cuál librería usar para un caso en particular, algunas pueden no satisfacer del todo la necesidad que tiene planteada. Asegúrese de saber el lenguaje de programación en el que se encuentra la librería. Si lo que necesita es hacer un bosquejo inicial de forma rápida, elija la librería que le brinda gráficos más detallados con el menor esfuerzo.

Recuerde que la visualización interactiva le permite construir *dashboards* en los cuales puede visualizar más información. Pero no olvide que una buena visualización no es la que contiene mucha información, sino aquella que hace énfasis en los datos que agregan valor al negocio.

Referencias

Plotly, Inc. Plotly Express in Python, (n.d.), Recuperado el 14 de enero de 2024 de https://plotly.com/python/plotly-express/

Anaconda, Inc., and Bokeh Contributors, Bokeh First steps, (n.d.), Recuperado el 15 de enero de 2024 de https://docs.bokeh.org/en/latest/docs/first_steps.html#first-steps

Novous Partners, NVD3 Reusable charts for d3.js, Recuperado el 15 de enero de 2024 de https://nvd3.org/

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Connor Gramazio, et al., 2014, Data Driven Document Introduction, Recuperado el 15 de enero de 2024 de https://d3js.org/what-is-d3

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

eig Business School | UIDE Powered by Arizona State University

VISUALIZACIÓN INTERACTIVA DE DATOS

¿Cúando usar gráficos 3D?

- Se requiere mostrar relacionescomplejas entre datos multidimencionales.
- Es importante respresentar formas, profundidades y patrones espaciales.
- Se busca simplicidad y claridad; los gráficos 2D suelen ser más legibles.
- Los datos pueden interpretarse fácilmente en dos dimensiones.

¿Cúando optar por gráficos interactivos?

- El usuario necesita explorar los datos y descubrir patrones por si mismo.
- Se requiere presentar una gran cantidad de información de forma accesible.
- El mensaje es simple y puede ser comunicado con un gráfico estático.
- La interactividad no añade valor significativo a la comprensión de los datos.

Elección de herramientas

Plotly
Ideal para gráficos interactivos y visualizaciones 3D dinámicas. Fácil intrepretación con Python.

Bokeh
Excelente para crear visulizaciones interactivas sofisticadas. Bueno para grandes conjuntos de datos.

NVD3
Basado en D3.js, es una buena opción para quienes prefieren trabajar JaveScript. Ofrece interactividad con un enfoque en reutilización de código.

Reglas de diseño para visualizaciones

1. Claridad:
Mantener un diseño limpio y no sobrecargar de información.

2. Consistencia:
Usar colores y estilos consistentes para evitar confusiones.

3. Enfoque:
Destacar los elementos más importantes de los datos.

4. Accesibilidad:
Asegurarse de que las visualizaciones sean comprensibles para todos los usuarios.

Recordatorio final:

1. Elegir en función del objetivo y del público.
2. Priorizar la claridad y la eficacia de la comunicación de datos.
3. Experimentar y aprender a través de la práctica.